

```
# =====
# 文件: app.py
# =====
"""
=====

文件名: app.py
作者: zjy / sbw
功能: 场馆预约系统后端API服务
描述: 提供用户注册/登录, 场地管理, 预约/取消/签到,
      推荐算法, 全局优化等完整API接口
=====
"""

from flask import Flask, jsonify, request, send_from_directory
from flask_cors import CORS
import sqlite3
import datetime
import re
import time
import threading
import os
from functools import wraps

from sa_duiqi import optimize as sa_optimize

app = Flask(__name__)
CORS(app)
app.config['JSON_AS_ASCII'] = False

# ----- 静态页面路由 -----
@app.route('/login.html')
def login_page():
    return send_from_directory(os.path.dirname(os.path.abspath(__file__)), 'login.html')

@app.route('/courts.html')
def courts_page():
    return send_from_directory(os.path.dirname(os.path.abspath(__file__)), 'courts.html')

@app.route('/my.html')
def my_page():
    return send_from_directory(os.path.dirname(os.path.abspath(__file__)), 'my.html')

@app.route('/admin.html')
def admin_page():
    return send_from_directory(os.path.dirname(os.path.abspath(__file__)), 'admin.html')

# =====
# 1. 辅助函数
# =====
```

```
def success_response(data=None, msg="操作成功"):
    """统一成功返回格式"""
    return jsonify({"success": True, "data": data, "msg": msg})

def error_response(msg="操作失败", status_code=400):
    """统一失败返回格式"""
    return jsonify({"success": False, "data": None, "msg": msg}), status_code

def log_request(func):
    """请求日志装饰器：记录时间、IP、方法和路径"""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        ip = request.remote_addr
        method = request.method
        path = request.path
        try:
            response = func(*args, **kwargs)
            status = response.status_code if hasattr(response, 'status_code') else 'OK'
        except Exception as e:
            status = f'ERROR: {str(e)}'
            raise
        print(f"[{start_time}] IP: {ip} | {method} {path} | 状态: {status}")
        return response
    return wrapper

def get_db():
    """获取数据库连接，返回字典格式数据"""
    import os
    db_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'court_booking.db')
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    return conn

def validate_court_id(court_id, cursor):
    """校验场地是否存在"""
    cursor.execute('SELECT id FROM courts WHERE id = ?', (court_id,))
    return cursor.fetchone() is not None

def validate_user_id(user_id, cursor):
    """校验用户是否存在"""
    cursor.execute('SELECT id FROM users WHERE id = ?', (user_id,))
    return cursor.fetchone() is not None
```

```
def validate_date(date_str):
    """校验日期格式 YYYY-MM-DD"""
    try:
        datetime.datetime.strptime(date_str, '%Y-%m-%d')
        return True
    except ValueError:
        return False

def validate_time(time_str):
    """校验时间格式 HH:MM"""
    try:
        datetime.datetime.strptime(time_str, '%H:%M')
        return True
    except ValueError:
        return False

def check_admin(request):
    """检查请求是否是管理员"""
    token = request.headers.get('Authorization', '')
    if not token:
        token = request.args.get('token', '')
    if not token:
        return False, None, '缺少 token'
    if token.startswith('user_'):
        try:
            user_id = int(token.split('_')[1])
        except (IndexError, ValueError):
            return False, None, 'token 格式错误'
    else:
        return False, None, 'token 格式错误'
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('SELECT role FROM users WHERE id = ?', (user_id,))
    row = cursor.fetchone()
    conn.close()
    if not row:
        return False, None, '用户不存在'
    if row['role'] != '管理员':
        return False, user_id, '权限不足, 需要管理员身份'
    return True, user_id, 'OK'

def extract_hour(time_str):
    """从时间字符串提取小时数字, 如 '18:00' -> 18"""
    if not time_str:
        return None
    match = re.search(r'(\d+)', str(time_str))
```

```

        return int(match.group(1)) if match else None

def extract_floor(location):
    """从位置字符串提取楼层数字，支持中文数字和阿拉伯数字"""
    if not location:
        return 1
    chinese_to_num = {'一': 1, '二': 2, '三': 3, '四': 4, '五': 5, '六': 6, '七': 7, '八': 8, '九': ...
    for ch, num in chinese_to_num.items():
        if ch in str(location):
            return num
    match = re.search(r'(\d+)', str(location))
    return int(match.group(1)) if match else 1

def release_overdue_reservations():
    """
    超时释放逻辑：
    检查所有状态为'预约中'的记录，若创建时间超过15分钟未签到，
    自动改为'已超时'并释放对应场地
    """
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT id, court_id, created_at
        FROM reservations
        WHERE status = '预约中'
        AND datetime(created_at, '+15 minutes') < datetime('now', 'localtime')
    ''')
    overdue_list = cursor.fetchall()
    for row in overdue_list:
        cursor.execute("UPDATE reservations SET status = '已超时' WHERE id = ?", (row['id'],))...
        cursor.execute("UPDATE courts SET status = '空闲' WHERE id = ?", (row['court_id'],))
    if overdue_list:
        conn.commit()
    conn.close()

# =====
# 2. API 接口
# =====

# ----- 2.1 获取场地列表 -----
@app.route('/api/courts', methods=['GET'])
@log_request
def get_courts():
    """获取所有场地的列表，包含 id、名称、类型、位置、状态"""
    release_overdue_reservations()
    conn = get_db()
    cursor = conn.cursor()

```

```
        cursor.execute('SELECT id, name, type, location, status FROM courts')
        courts = cursor.fetchall()
        conn.close()
        return success_response([dict(row) for row in courts])

# ----- 2.2 用户注册 -----
@app.route('/api/register', methods=['POST'])
@log_request
def register():
    """用户注册，需要提供 username 和 password，用户名至少3字符，密码至少4字符"""
    data = request.get_json()
    if not data:
        return error_response("请求体不能为空")
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return error_response("用户名和密码不能为空")
    if len(username) < 3:
        return error_response("用户名至少3个字符")
    if len(password) < 4:
        return error_response("密码至少4个字符")

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('SELECT id FROM users WHERE username = ?', (username,))
    if cursor.fetchone():
        conn.close()
        return error_response("用户名已存在", 400)

    invite_code = data.get('invite_code', '')
    role = '管理员' if invite_code == 'admin666' else '普通用户'
    cursor.execute('INSERT INTO users (username, password, role) VALUES (?, ?, ?)',
                  (username, password, role))

    conn.commit()
    user_id = cursor.lastrowid
    conn.close()
    return success_response({"user_id": user_id, "role": role}, f"注册成功, 角色: {role}")

# ----- 2.3 用户登录 -----
@app.route('/api/login', methods=['POST'])
@log_request
def login():
    """用户登录，验证用户名和密码，返回 token 和用户信息"""
    data = request.get_json()
    if not data:
        return error_response("请求体不能为空")
    username = data.get('username')
```

```

password = data.get('password')

if not username or not password:
    return error_response("用户名和密码不能为空")

conn = get_db()
cursor = conn.cursor()
cursor.execute('SELECT id, username, role FROM users WHERE username=? AND password=?',
               (username, password))
user = cursor.fetchone()
conn.close()

if not user:
    return error_response("用户名或密码错误", 401)

return success_response({
    "token": f'user_{user["id"]}',
    "user_id": user["id"],
    "role": user["role"]
}, "登录成功")

# ----- 2.4 创建预约（并发安全版） -----
@app.route('/api/book', methods=['POST'])
@log_request
def book COURT():
    """
    创建预约，需要提供 user_id, court_id, date, start_time, end_time
    会校验参数格式、用户和场地是否存在、时段是否冲突
    使用数据库锁保证并发安全
    """
    data = request.get_json()
    if not data:
        return error_response("请求体不能为空")

    user_id = data.get('user_id')
    court_id = data.get('court_id')
    date = data.get('date')
    start_time = data.get('start_time')
    end_time = data.get('end_time')

    # 参数完整性校验
    if not all([user_id, court_id, date, start_time, end_time]):
        return error_response("缺少必填参数: user_id, court_id, date, start_time, end_time")

    # 格式校验
    if not validate_time(start_time):
        return error_response(f"start_time 格式错误, 应为 HH:MM (当前值: {start_time})")
    if not validate_time(end_time):
        return error_response(f"end_time 格式错误, 应为 HH:MM (当前值: {end_time})")

```

```

    if start_time >= end_time:
        return error_response("start_time 必须早于 end_time")
    if not validate_date(date):
        return error_response(f"date 格式错误, 应为 YYYY-MM-DD (当前值: {date})")

    conn = get_db()
    cursor = conn.cursor()

    # 用户和场地存在性校验
    if not validate_user_id(user_id, cursor):
        conn.close()
        return error_response(f"user_id {user_id} 不存在")
    if not validate_court_id(court_id, cursor):
        conn.close()
        return error_response(f"court_id {court_id} 不存在")

    # ===== 并发安全: 显式加锁 =====
    cursor.execute('BEGIN IMMEDIATE')
    try:
        # 时段冲突检查 (在锁内重新检查, 确保读取的是最新数据)
        cursor.execute('''
            SELECT id FROM reservations
            WHERE court_id = ? AND date = ? AND status != '已取消' AND status != '已超时'
            AND ((start_time <= ? AND end_time > ?) OR (start_time < ? AND end_time >= ?...
        ''', (court_id, date, start_time, start_time, end_time, end_time, start_time, end...

        conflict = cursor.fetchone()
        if conflict:
            conn.rollback()
            conn.close()
            return error_response("该场地刚被他人预约, 请刷新后重试", 409)

        # 插入预约记录
        cursor.execute('''
            INSERT INTO reservations (user_id, court_id, date, start_time, end_time, statu...
            VALUES (?, ?, ?, ?, ?, ?, datetime('now', 'localtime'))
        ''', (user_id, court_id, date, start_time, end_time, '预约中'))
        cursor.execute("UPDATE courts SET status = '预约中' WHERE id = ?", (court_id,))

        conn.commit()
        reservation_id = cursor.lastrowid
        conn.close()

        return success_response({"reservation_id": reservation_id}, "预约成功")

    except sqlite3.OperationalError as e:
        conn.rollback()
        conn.close()
        return error_response("系统繁忙, 请稍后重试", 503)
# ----- 2.5 取消预约 -----

```

```

@app.route('/api/cancel', methods=['POST'])
@log_request
def cancel_reservation():
    """取消预约，需要提供 reservation_id 和 user_id，只能取消自己的预约"""
    data = request.get_json()
    if not data:
        return error_response("请求体不能为空")

    reservation_id = data.get('reservation_id')
    user_id = data.get('user_id')

    if not reservation_id or not user_id:
        return error_response("缺少 reservation_id 或 user_id")

    conn = get_db()
    cursor = conn.cursor()

    if not validate_user_id(user_id, cursor):
        conn.close()
        return error_response(f"user_id {user_id} 不存在")

    cursor.execute('SELECT court_id, user_id, status FROM reservations WHERE id = ?', (reservation_id,))
    row = cursor.fetchone()
    if not row:
        conn.close()
        return error_response(f"reservation_id {reservation_id} 不存在", 404)

    if row['user_id'] != user_id:
        conn.close()
        return error_response("只能取消自己的预约", 403)

    if row['status'] == '已取消':
        conn.close()
        return error_response("该预约已取消", 400)

    court_id = row['court_id']
    cursor.execute("UPDATE reservations SET status = '已取消' WHERE id = ?", (reservation_id,))
    cursor.execute("UPDATE courts SET status = '空闲' WHERE id = ?", (court_id,))
    conn.commit()
    conn.close()

    return success_response(None, "已取消预约")

# ----- 2.6 签到 -----
@app.route('/api/checkin', methods=['POST'])
@log_request
def checkin():
    """签到，需要提供 reservation_id 和 user_id，只能签到自己的预约"""
    release_overdue_reservations()

```



```
data = request.get_json()
if not data:
    return error_response("请求体不能为空")

reservation_id = data.get('reservation_id')
user_id = data.get('user_id')

if not reservation_id or not user_id:
    return error_response("缺少 reservation_id 或 user_id")

conn = get_db()
cursor = conn.cursor()

if not validate_user_id(user_id, cursor):
    conn.close()
    return error_response(f"user_id {user_id} 不存在")

cursor.execute('SELECT court_id, user_id, status FROM reservations WHERE id = ?', (res...
row = cursor.fetchone()
if not row:
    conn.close()
    return error_response(f"reservation_id {reservation_id} 不存在", 404)

if row['user_id'] != user_id:
    conn.close()
    return error_response("只能签到自己的预约", 403)

if row['status'] != '预约中':
    conn.close()
    return error_response("该预约状态不是预约中, 无法签到", 400)

court_id = row['court_id']
cursor.execute("UPDATE reservations SET status = '已使用' WHERE id = ?", (reservation_id,...
cursor.execute("UPDATE courts SET status = '占用' WHERE id = ?", (court_id,))
conn.commit()
conn.close()

return success_response(None, "签到成功")

# ----- 2.7 根路径 -----
@app.route('/')
@log_request
def index():
    """根路径, 返回可用接口列表"""
    return success_response({
        "available_apis": [
            "/api/register",
            "/api/login",
```

```

        "/api/courts",
        "/api/book",
        "/api/cancel",
        "/api/checkin",
        "/api/my_reservations",
        "/api/recommend",
        "/api/users",
        "/api/optimize"
    ],
}, "场馆预约系统 API 已启动")

# ----- 2.8 查询我的预约 -----
@app.route('/api/my_reservations', methods=['GET'])
@log_request
def my_reservations():
    """查询当前用户的所有预约记录（不含已取消的）"""
    release_overdue_reservations()

    user_id = request.args.get('user_id')
    if not user_id:
        return error_response("缺少 user_id 参数")

    try:
        user_id = int(user_id)
    except ValueError:
        return error_response("user_id 必须为数字")

    conn = get_db()
    cursor = conn.cursor()

    if not validate_user_id(user_id, cursor):
        conn.close()
        return error_response(f"user_id {user_id} 不存在")

    cursor.execute('''
        SELECT r.id, r.court_id, c.name AS court_name, c.type AS court_type,
               c.location, r.date, r.start_time, r.end_time, r.status
        FROM reservations r
        JOIN courts c ON r.court_id = c.id
        WHERE r.user_id = ? AND r.status != '已取消'
        ORDER BY r.date DESC, r.start_time ASC
    ''', (user_id,))

    reservations = cursor.fetchall()
    conn.close()

    data = []
    for row in reservations:
        data.append({

```

```

        'id': row['id'],
        'court_id': row['court_id'],
        'court_name': row['court_name'],
        'court_type': row['court_type'],
        'location': row['location'],
        'date': row['date'],
        'start_time': row['start_time'],
        'end_time': row['end_time'],
        'status': row['status']
    })

    return success_response(data)

# ----- 2.9 推荐算法 -----
@app.route('/api/recommend', methods=['GET'])
@log_request
def recommend():
    """
    根据用户历史预约记录推荐场地
    权重: 类型匹配 50分, 楼层匹配 30分, 时段匹配 20分, 总分100
    """
    user_id = request.args.get('user_id')
    if not user_id:
        return error_response("缺少 user_id 参数")

    try:
        user_id = int(user_id)
    except ValueError:
        return error_response("user_id 必须为数字")

    conn = get_db()
    cursor = conn.cursor()

    if not validate_user_id(user_id, cursor):
        conn.close()
        return error_response(f"user_id {user_id} 不存在")

    # 获取用户偏好类型
    cursor.execute('''
        SELECT c.type, COUNT(*) as count
        FROM reservations r
        JOIN courts c ON r.court_id = c.id
        WHERE r.user_id = ? AND r.status != '已取消' AND r.status != '已超时'
        GROUP BY c.type
        ORDER BY count DESC LIMIT 1
    ''', (user_id,))
    result = cursor.fetchone()
    preferred_type = result['type'] if result else None

```

```

# 获取用户偏好时段
cursor.execute('''
    SELECT start_time, COUNT(*) as count
    FROM reservations
    WHERE user_id = ? AND status != '已取消' AND status != '已超时'
    GROUP BY start_time
    ORDER BY count DESC LIMIT 1
''', (user_id,))
result = cursor.fetchone()
preferred_time = result['start_time'] if result else None
preferred_hour = extract_hour(preferred_time) if preferred_time else None

# 获取用户偏好楼层
cursor.execute('''
    SELECT c.location, COUNT(*) as count
    FROM reservations r
    JOIN courts c ON r.court_id = c.id
    WHERE r.user_id = ? AND r.status != '已取消' AND r.status != '已超时'
    GROUP BY c.location
    ORDER BY count DESC LIMIT 1
''', (user_id,))
result = cursor.fetchone()
preferred_location = result['location'] if result else None
preferred_floor = extract_floor(preferred_location) if preferred_location else None

# 获取所有空闲场地
cursor.execute('SELECT id, name, type, location, status FROM courts')
courts = cursor.fetchall()
conn.close()

# 无历史偏好时返回空推荐
if preferred_type is None:
    recommendations = []
    for c in courts[:5]:
        recommendations.append({
            'id': c['id'],
            'name': c['name'],
            'type': c['type'],
            'location': c['location'],
            'status': c['status'],
            'score': 50,
            'reason': '空闲场地推荐'
        })
    return success_response({
        "preferred_type": None,
        "preferred_time": None,
        "preferred_floor": None,
        "recommendations": recommendations
    })

```

```
# 计算每个场地的推荐分数
scored_courts = []
for court in courts:
    score = 0
    reasons = []

    # 类型匹配 +50
    if court['type'] == preferred_type:
        score += 50
        reasons.append("类型匹配 +50")
    else:
        score += 5
        reasons.append("类型不匹配 +5")

    # 楼层匹配 +30
    court_floor = extract_floor(court['location'])
    if preferred_floor is not None:
        if court_floor == preferred_floor:
            score += 30
            reasons.append("同楼层 +30")
        else:
            distance = abs(court_floor - preferred_floor)
            floor_score = max(0, 30 - distance * 10)
            score += floor_score
            reasons.append(f"楼层近 +{floor_score}" if floor_score > 0 else "楼层远 +0")
    else:
        score += 5
        reasons.append("无楼层偏好 +5")

    # 时段匹配 +20
    if preferred_hour is not None:
        if preferred_hour in [18, 19, 20, 21]:
            score += 20
            reasons.append("时段偏好 +20")
        else:
            score += 10
            reasons.append("时段一般 +10")
    else:
        score += 5
        reasons.append("无时段偏好 +5")

    final_score = min(score, 100)
    scored_courts.append({
        'id': court['id'],
        'name': court['name'],
        'type': court['type'],
        'location': court['location'],
        'status': court['status'],
        'score': final_score,
        'reason': '；'.join(reasons)
    })
```

```

    })

    scored_courts.sort(key=lambda x: x['score'], reverse=True)

    return success_response({
        "preferred_type": preferred_type,
        "preferred_time": preferred_time,
        "preferred_floor": preferred_floor,
        "recommendations": scored_courts[:5]
    })

# ----- 2.10 获取用户列表 -----
@app.route('/api/users', methods=['GET'])
@log_request
def get_users():
    """获取所有普通用户列表及其偏好类型（仅管理员可访问）"""
    # 权限验证
    is_admin, _, msg = check_admin(request)
    if not is_admin:
        return error_response(msg, 403)

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('SELECT id, username, role FROM users WHERE role = ?', ('普通用户',))
    users = cursor.fetchall()

    result = []
    for user in users:
        cursor.execute('''
            SELECT c.type, COUNT(*) as count
            FROM reservations r
            JOIN courts c ON r.court_id = c.id
            WHERE r.user_id = ? AND r.status != '已取消' AND r.status != '已超时'
            GROUP BY c.type
            ORDER BY count DESC LIMIT 1
        ''', (user['id'],))
        pref_row = cursor.fetchone()
        preference = pref_row['type'] if pref_row else '羽毛球'
        result.append({
            'id': user['id'],
            'username': user['username'],
            'preference': preference
        })
    conn.close()
    return success_response(result)

# ----- 2.11 全局优化 -----
@app.route('/api/optimize', methods=['POST'])

```

```
@log_request
def optimize():
    """
    全局优化分配：调用模拟退火算法，返回最优场地分配方案
    需要传入 courts 和 users 列表
    """

    # 权限验证：仅管理员可执行
    is_admin, _, msg = check_admin(request)
    if not is_admin:
        return error_response(msg, 403)

    data = request.get_json()
    if not data:
        return error_response("请求体不能为空")

    courts = data.get('courts', [])
    users = data.get('users', [])

    if not courts or not users:
        return error_response("缺少 courts 或 users 参数")

    if not isinstance(courts, list) or not isinstance(users, list):
        return error_response("courts 和 users 必须为数组")

    # 限制数据量，保证 3 秒内返回
    MAX_COURTS = 50
    MAX_USERS = 30
    if len(courts) > MAX_COURTS:
        courts = courts[:MAX_COURTS]
    if len(users) > MAX_USERS:
        users = users[:MAX_USERS]

    sa_result = {'data': None}
    error = {'msg': None}

    def run_optimize():
        try:
            sa_result['data'] = sa_optimize(
                courts=courts,
                users=users,
                preferences={
                    'initial_temperature': 100.0,
                    'min_temperature': 0.01,
                    'max_iterations': 10000,
                    'iterations_per_temp': 30,
                    'alpha': 0.95,
                    'verbose': False
                }
            )
        except Exception as e:
```

```

        error['msg'] = str(e)

    thread = threading.Thread(target=run_optimize)
    thread.start()
    thread.join(timeout=5.0)

    if thread.is_alive():
        return error_response("优化算法运行超时，请减少场地或用户数量后重试", 408)

    if error['msg']:
        return error_response(f"优化失败: {error['msg']}", 500)

    result = sa_result['data']
    if not result:
        return error_response('优化结果为空', 500)

    # success=False 仅表示存在少量场地冲突（如用户>场地时必然发生），
    # 不代表算法失败，仍返回分配方案供管理员参考
    if not result.get('plan'):
        return error_response(result.get('msg', '优化算法返回异常'), 500)

    # ===== 计算实际匹配率 =====
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT r.user_id, c.type AS court_type
        FROM reservations r
        JOIN courts c ON r.court_id = c.id
        WHERE r.status IN ('预约中', '已使用')
    ''')

    active_reservations = cursor.fetchall()
    actual_matched = 0
    actual_total = len(active_reservations)
    for res in active_reservations:
        user_id = res['user_id']
        court_type = res['court_type']
        cursor.execute('''
            SELECT c.type, COUNT(*) AS count
            FROM reservations r
            JOIN courts c ON r.court_id = c.id
            WHERE r.user_id = ? AND r.status != '已取消' AND r.status != '已超时'
            GROUP BY c.type ORDER BY count DESC LIMIT 1
        ''', (user_id,))
        pref_row = cursor.fetchone()
        preference = pref_row['type'] if pref_row else ''
        if preference == court_type:
            actual_matched += 1
    actual_match_rate = actual_matched / actual_total if actual_total > 0 else 0.0
    conn.close()

```



```

# 保险：算法结果如果比现状差，不展示分配方案
algorithm_rate = result.get('match_rate', 0)
if algorithm_rate < actual_match_rate:
    return success_response({
        "plan": [],
        "match_rate": algorithm_rate,
        "actual_match_rate": actual_match_rate,
        "actual_matched": actual_matched,
        "actual_total": actual_total,
        "runtime_seconds": result.get('runtime_seconds', 0),
        "total_iterations": result.get('total_iterations', 0),
        "msg": f"当前实际匹配率 ({actual_match_rate*100:.1f}%) 已优于算法建议 ({algorithm_rate*100:.1f}%)
    })

return success_response({
    "plan": result['plan'],
    "fitness": result.get('fitness', 0),
    "match_rate": result.get('match_rate', 0),
    "actual_match_rate": actual_match_rate,
    "actual_matched": actual_matched,
    "actual_total": actual_total,
    "runtime_seconds": result.get('runtime_seconds', 0),
    "total_iterations": result.get('total_iterations', 0)
})

# =====
# 3. 启动服务
# =====

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True, port=5000, use_reloader=False)
# =====
# 文件：sa_duiqi.py
# =====
"""
=====
文件名：sa_duiqi.py
作者：zrd
功能：模拟退火算法 - 场馆分配优化
描述：带时间维度和场地唯一性约束的优化算法，
        提供全局最优场地分配方案计算
=====
"""

import random
import time
import json
import sys
import os

```

```
from typing import List, Dict, Any, Tuple, Optional, Set
from datetime import datetime, timedelta
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
```

```
class SolutionCache:
    """解决方案缓存, 优化适应度计算"""
    def __init__(self):
        self.cache = {}
        self.hits = 0
        self.misses = 0

    def get(self, key):
        if key in self.cache:
            self.hits += 1
            return self.cache[key]
        self.misses += 1
        return None

    def set(self, key, value):
        self.cache[key] = value

    def clear(self):
        self.cache.clear()
        self.hits = 0
        self.misses = 0

def generate_test_data_with_time(
    num_courts: int = 20,
    num_users: int = 50,
    num_time_slots: int = 4,
    court_types: List[str] = None,
    seed: int = 42
) -> Tuple[List[Dict], List[Dict], List[Dict]]:
    """
    生成带时间维度的测试数据

    Args:
        num_courts: 场地数量
        num_users: 用户数量
        num_time_slots: 每个场地的时间段数
        court_types: 场地类型列表
        seed: 随机种子

    Returns:
        (courts, users, time_slots)
    """
```

```

random.seed(seed)
np.random.seed(seed)

if court_types is None:
    court_types = ["羽毛球", "乒乓球", "篮球", "足球", "网球", "排球", "游泳", "健身"]

time_slots = []
start_hour = 8
end_hour = 22
slot_duration = 1.5 # 小时

for i in range(num_time_slots):
    start_time = datetime(2026, 1, 1, start_hour, 0) + timedelta(hours=i * slot_durati...
    end_time = start_time + timedelta(hours=slot_duration)
    if end_time.hour >= end_hour:
        break
    time_slots.append({
        "slot_id": i + 1,
        "date": "2026-01-15",
        "start_time": start_time.strftime("%H:%M"),
        "end_time": end_time.strftime("%H:%M"),
        "index": i
    })

courts = []
court_id = 1
for i in range(num_courts):
    court_type = random.choice(court_types)
    location = f"{chr(65 + i % 5)}区"

    for slot in time_slots:
        courts.append({
            "id": court_id,
            "court_name": f"{court_type}场地{chr(65 + i)}",
            "type": court_type,
            "location": location,
            "slot_id": slot["slot_id"],
            "date": slot["date"],
            "start_time": slot["start_time"],
            "end_time": slot["end_time"],
            "time_index": slot["index"],
            "court_index": i,
            "unique_key": f"court_{i}_slot_{slot['slot_id']}"
        })
        court_id += 1

users = []
for i in range(num_users):
    num_prefs = random.choices([0, 1, 2], weights=[0.2, 0.5, 0.3])[0]
    if num_prefs > 0:
        prefs = random.sample(court_types, min(num_prefs, len(court_types)))
    else:

```

```

        prefs = []

        time_prefs = random.sample(
            [s["slot_id"] for s in time_slots],
            min(random.randint(1, 2), len(time_slots)))
    ) if len(time_slots) > 0 else []

    users.append({
        "id": i + 1,
        "name": f"用户{i+1}",
        "preference": prefs if len(prefs) > 1 else (prefs[0] if prefs else ""),
        "time_preferences": time_prefs,
        "level": random.choice(["初级", "中级", "高级"]),
        "priority": random.randint(1, 5)
    })

    return courts, users, time_slots

def calculate_fitness_with_constraints(
    solution: List[int],
    user_prefs: List[List[str]],
    court_info: List[Dict],
    user_time_prefs: List[List[int]] = None,
    cache: SolutionCache = None
) -> float:
    """
    计算适应度，包含场地唯一性约束

    Args:
        solution: 每个用户分配的场地索引（court_slot索引）
        user_prefs: 每个用户的偏好类型列表
        court_info: 场地信息列表
        user_time_prefs: 每个用户偏好的时间段
        cache: 缓存对象

    Returns:
        适应度值（越高越好）
    """
    if cache is not None:
        key = tuple(solution)
        cached = cache.get(key)
        if cached is not None:
            return cached

    num_users = len(solution)

    court_slot_usage = defaultdict(int)
    for court_idx in solution:
        court_slot_usage[court_idx] += 1

```

```
violation_penalty = 0
for court_idx, count in court_slot_usage.items():
    if count > 1:
        violation_penalty += (count - 1) * 10

if any(count > 1 for count in court_slot_usage.values()):
    violation_penalty += 50

match_score = 0
type_match_count = 0
time_match_count = 0

for user_idx, court_idx in enumerate(solution):
    if court_idx >= len(court_info):
        continue

    court = court_info[court_idx]
    court_type = court.get('type', '')

    user_pref_list = user_prefs[user_idx] if user_idx < len(user_prefs) else []
    if court_type in user_pref_list:
        type_match_count += 1
        match_score += 10

    if user_time_prefs and user_idx < len(user_time_prefs):
        user_time_prefs_list = user_time_prefs[user_idx]
        if user_time_prefs_list:
            slot_id = court.get('slot_id', 0)
            if slot_id in user_time_prefs_list:
                time_match_count += 1
                match_score += 5

no_pref_penalty = 0
for user_idx, court_idx in enumerate(solution):
    if court_idx < len(court_info):
        court = court_info[court_idx]
        court_type = court.get('type', '')
        user_pref_list = user_prefs[user_idx] if user_idx < len(user_prefs) else []
        if user_pref_list and court_type not in user_pref_list:
            no_pref_penalty += 2

# 4. 最终适应度
fitness = match_score - violation_penalty - no_pref_penalty

if cache is not None:
    cache.set(key, fitness)

return fitness
```

```
def count_matches_with_constraints(
    solution: List[int],
    user_prefs: List[List[str]],
    court_info: List[Dict],
    user_time_prefs: List[List[int]] = None
) -> Dict[str, Any]:
    """
    统计匹配情况
    """
    num_users = len(solution)

    court_slot_usage = defaultdict(list)
    for user_idx, court_idx in enumerate(solution):
        court_slot_usage[court_idx].append(user_idx)

    violations = 0
    for court_idx, users in court_slot_usage.items():
        if len(users) > 1:
            violations += len(users) - 1

    type_matches = 0
    time_matches = 0

    for user_idx, court_idx in enumerate(solution):
        if court_idx >= len(court_info):
            continue

        court = court_info[court_idx]
        court_type = court.get('type', '')

        user_pref_list = user_prefs[user_idx] if user_idx < len(user_prefs) else []
        if court_type in user_pref_list:
            type_matches += 1

        if user_time_prefs and user_idx < len(user_time_prefs):
            user_time_prefs_list = user_time_prefs[user_idx]
            if user_time_prefs_list:
                slot_id = court.get('slot_id', 0)
                if slot_id in user_time_prefs_list:
                    time_matches += 1

    matched_users = type_matches
    match_rate = matched_users / num_users if num_users > 0 else 0

    return {
        'matched_users': matched_users,
        'match_rate': match_rate,
        'type_matches': type_matches,
        'time_matches': time_matches,
```

```
        'violations': violations,
        'total_users': num_users,
        'court_usage': dict(court_slot_usage)
    }

def create_solution_with_constraints(
    num_users: int,
    num_court_slots: int,
    user_prefs: List[List[str]],
    court_info: List[Dict],
    user_time_prefs: List[List[int]] = None
) -> List[int]:
    """
    创建初始解（考虑场地唯一性）
    """
    solution = []
    used_courts = set()

    users_with_prefs = []
    users_without_prefs = []

    for user_idx in range(num_users):
        if user_prefs[user_idx]:
            users_with_prefs.append(user_idx)
        else:
            users_without_prefs.append(user_idx)

    for user_idx in users_with_prefs:
        prefs = user_prefs[user_idx]

        matching_courts = []
        for court_idx, court in enumerate(court_info):
            if court_idx in used_courts:
                continue
            if court.get('type', '') in prefs:
                matching_courts.append(court_idx)

        if matching_courts:
            if user_time_prefs and user_time_prefs[user_idx]:
                time_prefs = user_time_prefs[user_idx]
                time_matching = []
                for court_idx in matching_courts:
                    if court_info[court_idx].get('slot_id', 0) in time_prefs:
                        time_matching.append(court_idx)
            if time_matching:
                court_idx = random.choice(time_matching)
                used_courts.add(court_idx)
                solution.append(court_idx)
            continue
```

```
        court_idx = random.choice(matching_courts)
        used_courts.add(court_idx)
        solution.append(court_idx)
    else:
        available = [i for i in range(num_court_slots) if i not in used_courts]
        if available:
            court_idx = random.choice(available)
            used_courts.add(court_idx)
            solution.append(court_idx)
        else:
            solution.append(random.randint(0, num_court_slots - 1))

for user_idx in users_without_prefs:
    available = [i for i in range(num_court_slots) if i not in used_courts]
    if available:
        court_idx = random.choice(available)
        used_courts.add(court_idx)
        solution.append(court_idx)
    else:
        solution.append(random.randint(0, num_court_slots - 1))

return solution


def optimize_with_time(
    courts: List[Dict],
    users: List[Dict],
    time_slots: List[Dict] = None,
    preferences: Optional[Dict[str, Any]] = None
) -> Dict[str, Any]:
    """
    优化的模拟退火算法 - 带时间维度和场地唯一性约束

    Args:
        courts: 场地列表（每个场地包含时间段信息）
        users: 用户列表
        time_slots: 时间段列表
        preferences: 算法参数

    Returns:
        优化结果
    """
    if preferences is None:
        preferences = {}

    config = {
        'initial_temperature': preferences.get('initial_temperature', 80.0),
        'min_temperature': preferences.get('min_temperature', 0.1),
        'alpha': preferences.get('alpha', 0.88),
```



```

        'iterations_per_temp': preferences.get('iterations_per_temp', 60),
        'max_iterations': preferences.get('max_iterations', 80000),
        'verbose': preferences.get('verbose', False),
        'random_seed': preferences.get('random_seed', 42),
        'max_no_improvement': preferences.get('max_no_improvement', 1000)
    }

    if config['random_seed'] is not None:
        random.seed(config['random_seed'])
        np.random.seed(config['random_seed'])

    num_court_slots = len(courts)
    num_users = len(users)

    user_prefs = []
    user_time_prefs = []
    for user in users:
        pref = user.get('preference', '')
        if isinstance(pref, str):
            user_prefs.append([pref] if pref else [])
        else:
            user_prefs.append(pref if pref else [])

        time_pref = user.get('time_preferences', [])
        if isinstance(time_pref, (int, str)):
            user_time_prefs.append([time_pref] if time_pref else [])
        else:
            user_time_prefs.append(time_pref if time_pref else [])

    court_info = []
    for court in courts:
        court_info.append({
            'type': court.get('type', ''),
            'location': court.get('location', ''),
            'slot_id': court.get('slot_id', 0),
            'time_index': court.get('time_index', 0),
            'court_index': court.get('court_index', 0),
            'court_name': court.get('court_name', ''),
            'date': court.get('date', ''),
            'start_time': court.get('start_time', ''),
            'end_time': court.get('end_time', ''),
            'unique_key': court.get('unique_key', '')
        })

    solution = create_solution_with_constraints(
        num_users, num_court_slots, user_prefs, court_info, user_time_prefs
    )

    while len(solution) < num_users:
        available = [i for i in range(num_court_slots)]

```

```

        if available:
            solution.append(random.choice(available))
        else:
            solution.append(random.randint(0, num_court_slots - 1))

cache = SolutionCache()
initial_fitness = calculate_fitness_with_constraints(
    solution, user_prefs, court_info, user_time_prefs, cache
)

best_solution = solution.copy()
best_fitness = initial_fitness
current_solution = solution
current_fitness = initial_fitness

temperature = config['initial_temperature']
min_temperature = config['min_temperature']
alpha = config['alpha']
iterations_per_temp = config['iterations_per_temp']
max_iterations = config['max_iterations']
max_no_improvement = config['max_no_improvement']

total_iterations = 0
no_improvement_count = 0
best_fitness_history = [best_fitness]

if config['verbose']:
    print(f"初始适应度: {initial_fitness:.4f}")
    print(f"场地-时间段组合数: {num_court_slots}, 用户数: {num_users}")

while temperature > min_temperature and total_iterations < max_iterations:
    for _ in range(iterations_per_temp):
        if total_iterations >= max_iterations:
            break

        total_iterations += 1

        new_solution = current_solution.copy()

        user_idx = random.randint(0, num_users - 1)
        new_court = random.randint(0, num_court_slots - 1)
        new_solution[user_idx] = new_court

        if random.random() < 0.3 and num_users >= 2:
            user1 = random.randint(0, num_users - 1)
            user2 = random.randint(0, num_users - 1)
            if user1 != user2:
                new_solution[user1], new_solution[user2] = new_solution[user2], new_so...

        new_fitness = calculate_fitness_with_constraints(

```

```

        new_solution, user_prefs, court_info, user_time_prefs, cache
    )

    if new_fitness > current_fitness:
        current_solution = new_solution
        current_fitness = new_fitness

        if new_fitness > best_fitness:
            best_solution = new_solution.copy()
            best_fitness = new_fitness
            no_improvement_count = 0
            best_fitness_history.append(best_fitness)
        else:
            no_improvement_count += 1
    else:
        delta = new_fitness - current_fitness
        if delta < 0 and random.random() < np.exp(delta / temperature):
            current_solution = new_solution
            current_fitness = new_fitness
            no_improvement_count += 1
        else:
            no_improvement_count += 1

    if no_improvement_count > max_no_improvement and temperature < 20:
        if config['verbose']:
            print(f"提前终止: 无改进 {no_improvement_count} 步")
        break

    temperature *= alpha

    if config['verbose'] and total_iterations % (iterations_per_temp * 5) == 0:
        print(f"温度: {temperature:.4f}, 当前适应度: {current_fitness:.4f}, 最优: {best_fitness...

stats = count_matches_with_constraints(
    best_solution, user_prefs, court_info, user_time_prefs
)

assignments = []
for user_idx, court_idx in enumerate(best_solution):
    if court_idx < len(court_info):
        court = court_info[court_idx]
        user = users[user_idx] if user_idx < len(users) else {}
        assignments.append({
            'user_id': user.get('id', user_idx + 1),
            'user_name': user.get('name', f'用户{user_idx+1}'),
            'court_name': court.get('court_name', ''),
            'court_type': court.get('type', ''),
            'location': court.get('location', ''),
            'date': court.get('date', ''),
            'start_time': court.get('start_time', ''),

```

```

        'end_time': court.get('end_time', ''),
        'slot_id': court.get('slot_id', 0),
        'court_index': court.get('court_index', 0),
        'is_matched': court.get('type', '') in user_prefs[user_idx] if user_idx < ...
    })

return {
    'best_solution': best_solution,
    'fitness': best_fitness,
    'match_rate': stats['match_rate'],
    'matched_users': stats['matched_users'],
    'total_users': stats['total_users'],
    'type_matches': stats['type_matches'],
    'time_matches': stats['time_matches'],
    'violations': stats['violations'],
    'total_iterations': total_iterations,
    'final_temperature': temperature,
    'success': stats['violations'] == 0 and best_fitness > 0,
    'assignments': assignments,
    'court_usage': stats['court_usage'],
    'fitness_history': best_fitness_history
}

def test_with_constraints():
    """测试带约束的算法"""
    print("=" * 80)
    print("测试带时间维度和场地唯一性约束的算法")
    print("=" * 80)

    test_scenarios = [
        {"num_courts": 5, "num_users": 8, "num_slots": 3, "desc": "小规模 (5场地, 8用户)"},
        {"num_courts": 10, "num_users": 20, "num_slots": 4, "desc": "中规模 (10场地, 20用户)"},
        {"num_courts": 20, "num_users": 50, "num_slots": 4, "desc": "目标规模 (20场地, 50用户)"},
    ]

    all_results = []

    for scenario in test_scenarios:
        print(f"\n{'='*60}")
        print(f"场景: {scenario['desc']}")
        print(f"{'='*60}")

        courts, users, time_slots = generate_test_data_with_time(
            num_courts=scenario['num_courts'],
            num_users=scenario['num_users'],
            num_time_slots=scenario['num_slots'],
            seed=42
        )

```

```

print(f"场地-时间段组合数: {len(courts)}")
print(f"用户数: {len(users)}")
print(f"时间段数: {len(time_slots)}")

config = {
    'initial_temperature': 80,
    'min_temperature': 0.1,
    'alpha': 0.88,
    'iterations_per_temp': 60,
    'max_iterations': 80000,
    'verbose': False,
    'random_seed': 42,
    'max_no_improvement': 1000
}

run_times = []
match_rates = []
fitnesses = []
violations_list = []

for run in range(3):
    start_time = time.time()
    result = optimize_with_time(courts, users, time_slots, config)
    elapsed = time.time() - start_time

    run_times.append(elapsed)
    match_rates.append(result['match_rate'])
    fitnesses.append(result['fitness'])
    violations_list.append(result['violations'])

    print(f"  Run {run+1}: {elapsed:.3f}s, 匹配率: {result['match_rate']:.2%}, "
          f"违规: {result['violations']}, 适应度: {result['fitness']:.2f}")

avg_runtime = np.mean(run_times)
avg_match_rate = np.mean(match_rates)
avg_fitness = np.mean(fitnesses)
avg_violations = np.mean(violations_list)

print(f"\n  平均运行时间: {avg_runtime:.3f}秒")
print(f"  平均匹配率: {avg_match_rate:.2%}")
print(f"  平均适应度: {avg_fitness:.2f}")
print(f"  平均违规数: {avg_violations:.1f}")

if avg_violations == 0:
    print("    场地唯一性约束满足!")
else:
    print(f"    存在 {avg_violations:.0f} 个场地违规分配")

all_results.append({
    'scenario': scenario['desc'],

```

```

        'avg_runtime': avg_runtime,
        'avg_match_rate': avg_match_rate,
        'avg_fitness': avg_fitness,
        'avg_violations': avg_violations,
        'details': result
    })

    return all_results

def test_original_problem():
    """测试原始问题场景 - 16人5场地"""
    print("\n" + "=" * 80)
    print("测试原始问题场景: 16人, 5个场地, 每个场地2个时间段")
    print("=" * 80)

    court_types = ["羽毛球", "乒乓球", "篮球"]

    courts, users, time_slots = generate_test_data_with_time(
        num_courts=5,
        num_users=16,
        num_time_slots=2,
        court_types=court_types,
        seed=42
    )

    for i, user in enumerate(users):
        if i % 3 == 0:
            user['preference'] = "羽毛球"
        elif i % 3 == 1:
            user['preference'] = "乒乓球"
        else:
            user['preference'] = ["羽毛球", "乒乓球"]

    print(f"\n场地-时间段组合数: {len(courts)} (5个场地 × 2个时间段)")
    print(f"用户数: {len(users)}")
    print(f"时间段: {time_slots}")

    print("\n场地信息:")
    for court in courts[:5]:
        print(f"    {court['court_name']} - {court['date']} {court['start_time']}-{court['en...

    config = {
        'initial_temperature': 80,
        'min_temperature': 0.1,
        'alpha': 0.88,
        'iterations_per_temp': 60,
        'max_iterations': 50000,
        'verbose': True,
        'random_seed': 42
    }

```

```

    }

    start_time = time.time()
    result = optimize_with_time(courts, users, time_slots, config)
    elapsed = time.time() - start_time

    print(f"\n{'='*60}")
    print("优化结果:")
    print(f"{'='*60}")
    print(f"运行时间: {elapsed:.3f} 秒")
    print(f"匹配率: {result['match_rate']:.2%}")
    print(f"匹配用户: {result['matched_users']}/{result['total_users']}")
    print(f"类型匹配: {result['type_matches']}")
    print(f"时间匹配: {result['time_matches']}")
    print(f"违规数: {result['violations']}")
    print(f"适应度: {result['fitness']:.2f}")
    print(f"迭代次数: {result['total_iterations']}")
    print(f"成功: {result['success']}")

    print("\n分配详情 (前10个):")
    for i, assignment in enumerate(result['assignments'][:10]):
        status = " " if assignment['is_matched'] else " "
        print(f" {status} {assignment['user_name']} -> {assignment['court_name']} "
              f"({assignment['court_type']}) {assignment['start_time']}-{assignment['end_t...

    print("\n场地使用情况:")
    for court_idx, users_list in result['court_usage'].items():
        if court_idx < len(courts):
            court = courts[court_idx]
            print(f" {court['court_name']} ({court['start_time']}-{court['end_time']}): "
                  f"{len(users_list)} 人")

    if result['violations'] == 0:
        print("\n 所有场地唯一性约束满足!")
    else:
        print(f"\n 存在 {result['violations']} 个违规分配")

    return result

def plot_results(results):
    """绘制结果图"""
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    ax1 = axes[0]
    scenarios = [r['scenario'] for r in results]
    times = [r['avg_runtime'] for r in results]
    rates = [r['avg_match_rate'] for r in results]

    x = np.arange(len(scenarios))

```

```

width = 0.35

bars1 = ax1.bar(x - width/2, times, width, label='运行时间 (秒)', color='steelblue', alpha=...
ax1.set_xlabel('场景')
ax1.set_ylabel('运行时间 (秒)')
ax1.set_title('算法性能')
ax1.set_xticks(x)
ax1.set_xticklabels(scenarios, rotation=15, ha='right')
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.3, axis='y')

ax2 = ax1.twinx()
bars2 = ax2.bar(x + width/2, rates, width, label='匹配率', color='coral', alpha=0.8)
ax2.set_ylabel('匹配率')
ax2.legend(loc='upper right')

ax3 = axes[1]
violations = [r['avg_violations'] for r in results]
fitness = [r['avg_fitness'] for r in results]

bars3 = ax3.bar(x - width/2, violations, width, label='违规数', color='red', alpha=0.7)
ax3.set_xlabel('场景')
ax3.set_ylabel('违规数')
ax3.set_title('约束满足情况')
ax3.set_xticks(x)
ax3.set_xticklabels(scenarios, rotation=15, ha='right')
ax3.legend(loc='upper left')
ax3.grid(True, alpha=0.3, axis='y')

ax4 = ax3.twinx()
bars4 = ax4.bar(x + width/2, fitness, width, label='适应度', color='green', alpha=0.7)
ax4.set_ylabel('适应度')
ax4.legend(loc='upper right')

plt.tight_layout()
plt.savefig('time_constraint_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n结果图已保存到: time_constraint_results.png")

def main():
    """主函数"""
    print("=" * 80)
    print("模拟退火算法 - 带时间维度和场地唯一性约束")
    print("=" * 80)

    print("\n【1】测试原始问题 (16人, 5场地, 每场地2时间段)")
    result = test_original_problem()

    print("\n【2】测试不同规模场景")

```

```

    results = test_with_constraints()

    plot_results(results)

    with open('time_constraint_results.json', 'w', encoding='utf-8') as f:
        json.dump({
            'original_problem': result,
            'scenarios': results
        }, f, indent=2, ensure_ascii=False, default=str)

    print("\n详细结果已保存到: time_constraint_results.json")

if __name__ == "__main__":
    main()

# ===== 兼容旧接口 =====

def optimize(
    courts: List[Dict[str, Any]],
    users: List[Dict[str, Any]],
    preferences: Optional[Dict[str, Any]] = None,
    cache = None
) -> Dict[str, Any]:
    """
    兼容旧接口的优化函数
    直接在场地上做分配（不展开假时间段），确保每个用户分配到不同场地
    """

    import time as _time
    start_time = _time.time()

    # 给原始 courts 补充算法需要的字段（单一时段，使用真实时间）
    time_templates = [
        ("18:00", "20:00"),
        ("19:00", "21:00"),
        ("20:00", "22:00"),
        ("18:30", "20:30"),
    ]

    court_slots = []
    for i, court in enumerate(courts):
        start_t, end_t = time_templates[i % len(time_templates)]
        court_slots.append({
            "id": court.get("id", i + 1),
            "court_name": court.get("name", f"场地{i+1}"),
            "type": court.get("type", ""),
            "location": court.get("location", ""),
            "slot_id": 1,
            "date": "2026-08-16",
            "start_time": start_t,

```

```

        "end_time": end_t,
        "time_index": 0,
        "court_index": i,
        "unique_key": f"court_{i}_slot_1",
        "original_court_id": court.get("id", i + 1)
    })

# 适配 users 格式
users_adapted = []
for user in users:
    u = dict(user)
    if "time_preferences" not in u:
        u["time_preferences"] = []
    if "name" not in u:
        u["name"] = u.get("username", f"用户{u.get('id', 0)}")
    users_adapted.append(u)

# 构造一个虚拟的 time_slots (只包含1个时段)
time_slots = [{
    "slot_id": 1,
    "date": "2026-08-16",
    "start_time": "18:00",
    "end_time": "20:00",
    "index": 0
}]

# 调用算法: 传入不固定的随机种子, 避免每次都一样
pref = dict(preferences) if preferences else {}
if 'random_seed' not in pref:
    pref['random_seed'] = None # 不固定种子

result = optimize_with_time(court_slots, users_adapted, time_slots, pref)

# 预先构建用户偏好映射
user_prefs_map = {}
for user in users:
    pref_val = user.get("preference", "")
    uid = user.get("id", 0)
    if isinstance(pref_val, str):
        user_prefs_map[uid] = [pref_val] if pref_val else []
    elif isinstance(pref_val, list):
        user_prefs_map[uid] = [p for p in pref_val if p]
    else:
        user_prefs_map[uid] = []

# 转换 assignments -> plan (兼容旧格式)
plan = []
matched_users = 0

for assignment in result.get("assignments", []):

```

```

# 直接从 assignment 取 court_index 映射回原始场地
court_idx = assignment.get("court_index", 0)
if court_idx < len(courts):
    original_court = courts[court_idx]
    court_id_val = original_court.get("id", court_idx + 1)
    court_type = original_court.get("type", "")
else:
    court_id_val = assignment.get("original_court_id", assignment.get("id", 0))
    court_type = assignment.get("court_type", "")

user_id = assignment.get("user_id", 0)
user_prefs = user_prefs_map.get(user_id, [])

is_matched = court_type in user_prefs if user_prefs else False
score = 1.0 if is_matched else 0.0
if is_matched:
    matched_users += 1

plan.append({
    "user_id": user_id,
    "court_id": court_id_val,
    "court_type": court_type,
    "user_preference": user_prefs,
    "score": score,
    "slot_id": 1,
    "start_time": assignment.get("start_time", ""),
    "end_time": assignment.get("end_time", "")
})

total_users = len(users)
match_rate = matched_users / total_users if total_users > 0 else 0.0

violations = result.get('violations', 0)
return {
    "success": True,
    "msg": f"优化完成, 总迭代次数: {result.get('total_iterations', 0)}, 存在 {violations} 个场地冲突",
    "plan": plan,
    "fitness": result.get("fitness", 0),
    "match_rate": match_rate,
    "matched_users": matched_users,
    "total_users": total_users,
    "runtime_seconds": _time.time() - start_time,
    "total_iterations": result.get("total_iterations", 0),
    "final_temperature": result.get("final_temperature", 0)
}

# =====
# 文件: neural_fitness.py
# =====
"""
=====

```

文件名: neural_fitness.py

作者: zrd

功能: 神经网络预测用户满意度 + 模拟退火优化

描述: 用PyTorch训练神经网络预测用户对场地的满意度,
将预测结果融入模拟退火的适应度函数

=====

"""

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import random
from collections import defaultdict
```

----- 1. 神经网络模型 -----

```
class SatisfactionNet(nn.Module):
```

"""

满意度预测网络

输入: [用户偏好类型one-hot(4维), 用户偏好时段(1维), 场地类型one-hot(4维), 场地时段(1维), 楼层差(1维)]

输出: 满意度分数 (0-1)

"""

```
def __init__(self, input_size=11, hidden_size=32):
    super(SatisfactionNet, self).__init__()
    self.net = nn.Sequential(
        nn.Linear(input_size, hidden_size),
        nn.ReLU(),
        nn.Dropout(0.1), # 防止过拟合
        nn.Linear(hidden_size, hidden_size // 2),
        nn.ReLU(),
        nn.Linear(hidden_size // 2, 1),
        nn.Sigmoid() # 输出0-1
    )
```

```
def forward(self, x):
    return self.net(x)
```

----- 2. 特征编码 -----

COURT_TYPES = ["羽毛球", "乒乓球", "篮球", "足球"]

```
def encode_features(user_pref_type, user_pref_hour, court_type, court_hour, floor_diff):
```

"""

将用户和场地特征编码为神经网络输入向量

Args:

user_pref_type: 用户偏好类型, 如 "羽毛球"

user_pref_hour: 用户偏好小时, 如 18

court_type: 场地类型, 如 "羽毛球"
 court_hour: 场地时段小时, 如 18
 floor_diff: 楼层差, 如 0 (同层) 或 1 (差一层)

Returns:

torch.Tensor: 11维特征向量
 """

用户偏好类型 one-hot (4维)

user_type_vec = [1 if t == user_pref_type else 0 for t in COURT_TYPES]

场地类型 one-hot (4维)

court_type_vec = [1 if t == court_type else 0 for t in COURT_TYPES]

数值特征归一化

user_hour_norm = (user_pref_hour - 8) / 14.0 if user_pref_hour else 0.5 # 8-22点归一化

court_hour_norm = (court_hour - 8) / 14.0

floor_diff_norm = min(abs(floor_diff) / 5.0, 1.0) # 楼层差归一化

features = user_type_vec + [user_hour_norm] + court_type_vec + [court_hour_norm, floor...
 return torch.FloatTensor(features)

----- 3. 生成训练数据 -----

def generate_training_data(reservations, courts, users):

"""

从历史预约记录生成训练样本

Args:

reservations: 预约记录列表

courts: 场地列表

users: 用户列表

Returns:

X: 特征矩阵

y: 满意度标签 (1=满意, 0=不满意)

"""

X = []

y = []

用户偏好映射

user_prefs = {}

for user in users:

pref = user.get('preference', '')

time_prefs = user.get('time_preferences', [])

user_prefs[user['id']] = {

'type': pref if isinstance(pref, str) and pref else random.choice(COURT_TYPES)...

'hour': time_prefs[0] if time_prefs else 18

}

```

# 场地特征映射
court_map = {c['id']: c for c in courts}

for res in reservations:
    user_id = res.get('user_id', 0)
    court_id = res.get('court_id', 0)
    status = res.get('status', '')

    if status in ('已取消', '已超时'):
        continue # 跳过取消的

    user_info = user_prefs.get(user_id, {'type': '羽毛球', 'hour': 18})
    court_info = court_map.get(court_id, {})

    if not court_info:
        continue

    court_type = court_info.get('type', '羽毛球')
    court_hour = extract_hour(court_info.get('start_time', '18:00'))
    user_floor = extract_floor(user_info.get('type', '')) # 简化处理
    court_floor = extract_floor(court_info.get('location', ''))

    features = encode_features(
        user_pref_type=user_info['type'],
        user_pref_hour=user_info['hour'],
        court_type=court_type,
        court_hour=court_hour,
        floor_diff=court_floor - user_floor
    )

    # 判断满意度: 类型匹配=满意
    is_satisfied = 1.0 if user_info['type'] == court_type else 0.0

    X.append(features)
    y.append(is_satisfied)

# 如果没有数据, 生成一些随机样本
if len(X) < 10:
    for _ in range(50):
        u_type = random.choice(COURT_TYPES)
        u_hour = random.randint(8, 22)
        c_type = random.choice(COURT_TYPES)
        c_hour = random.randint(8, 22)
        f_diff = random.randint(0, 3)

        features = encode_features(u_type, u_hour, c_type, c_hour, f_diff)
        is_satisfied = 1.0 if u_type == c_type else 0.0
        X.append(features)
        y.append(is_satisfied)

```

```
        return torch.stack(X), torch.FloatTensor(y).unsqueeze(1)

def extract_hour(time_str):
    """从时间字符串提取小时"""
    if not time_str:
        return 18
    try:
        return int(str(time_str).split(':')[0])
    except:
        return 18

def extract_floor(location):
    """从位置提取楼层"""
    if not location:
        return 1
    chinese_to_num = {'一': 1, '二': 2, '三': 3, '四': 4, '五': 5}
    for ch, num in chinese_to_num.items():
        if ch in str(location):
            return num
    import re
    match = re.search(r'(\d+)', str(location))
    return int(match.group(1)) if match else 1
```

----- 4. 训练模型 -----

```
def train_model(reservations, courts, users, epochs=100, lr=0.01):
    """
    训练满意度预测模型

    Returns:
        model: 训练好的模型
        history: 训练loss历史
    """
    X, y = generate_training_data(reservations, courts, users)

    model = SatisfactionNet()
    criterion = nn.BCELoss() # 二分类交叉熵
    optimizer = optim.Adam(model.parameters(), lr=lr)

    loss_history = []

    model.train()
    for epoch in range(epochs):
        optimizer.zero_grad()
        predictions = model(X)
        loss = criterion(predictions, y)
        loss.backward()
        optimizer.step()
```

```
        loss_history.append(loss.item())

    if (epoch + 1) % 20 == 0:
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}")

    return model, loss_history

# ----- 5. 预测接口 -----

def predict_satisfaction(model, user, court):
    """
    预测某个用户对某个场地的满意度

    Args:
        model: 训练好的神经网络
        user: 用户字典
        court: 场地字典

    Returns:
        float: 满意度分数 0-1
    """
    pref = user.get('preference', '')
    if isinstance(pref, list):
        pref = pref[0] if pref else '羽毛球'
    pref = pref if pref else '羽毛球'

    time_prefs = user.get('time_preferences', [])
    user_hour = time_prefs[0] if time_prefs else 18

    court_type = court.get('type', '羽毛球')
    court_hour = extract_hour(court.get('start_time', '18:00'))

    user_floor = extract_floor(user.get('location', ''))
    court_floor = extract_floor(court.get('location', ''))

    features = encode_features(pref, user_hour, court_type, court_hour, court_floor - user...

    model.eval()
    with torch.no_grad():
        score = model(features.unsqueeze(0)).item()

    return score

# ----- 6. 混合适应度函数 -----

def calculate_neural_fitness(solution, users, courts, model, alpha=0.6):
    """
```


神经网络 + 规则 混合适应度

```

alpha: 神经网络权重 (0-1)
"""

neural_score = 0
rule_score = 0
violations = 0

used_courts = set()

for user_idx, court_idx in enumerate(solution):
    if court_idx in used_courts:
        violations += 1
        continue
    used_courts.add(court_idx)

    user = users[user_idx]
    court = courts[court_idx]

    # 神经网络预测分数
    neural_score += predict_satisfaction(model, user, court)

    # 传统规则分数
    pref = user.get('preference', '')
    court_type = court.get('type', '')
    if pref == court_type:
        rule_score += 1

# 混合: 神经网络 + 规则
fitness = alpha * neural_score + (1 - alpha) * rule_score - violations * 10

return fitness

# ----- 7. 保存/加载模型 -----

def save_model(model, path='satisfaction_model.pth'):
    torch.save(model.state_dict(), path)
    print(f"模型已保存: {path}")

def load_model(path='satisfaction_model.pth'):
    model = SatisfactionNet()
    model.load_state_dict(torch.load(path, weights_only=True))
    model.eval()
    return model

# ----- 8. 测试 -----

if __name__ == "__main__":

```

```

# 生成测试数据
test_users = [
    {'id': 1, 'preference': '羽毛球', 'time_preferences': [18]},
    {'id': 2, 'preference': '篮球', 'time_preferences': [20]},
    {'id': 3, 'preference': '羽毛球', 'time_preferences': [18]},
]

test_courts = [
    {'id': 1, 'type': '羽毛球', 'start_time': '18:00', 'location': 'A区一楼'},
    {'id': 2, 'type': '篮球', 'start_time': '20:00', 'location': 'B区二楼'},
    {'id': 3, 'type': '羽毛球', 'start_time': '18:00', 'location': 'A区二楼'},
]

test_reservations = [
    {'user_id': 1, 'court_id': 1, 'status': '已使用'},
    {'user_id': 2, 'court_id': 2, 'status': '已使用'},
    {'user_id': 3, 'court_id': 3, 'status': '已使用'},
]

print("=" * 50)
print("训练满意度预测模型...")
print("=" * 50)

model, history = train_model(test_reservations, test_courts, test_users, epochs=50)

# 测试预测
print("\n预测测试:")
for user in test_users:
    for court in test_courts:
        score = predict_satisfaction(model, user, court)
        print(f"  用户{user['id']} ({user['preference']}) -> {court['type']}场地: {score:.3...}")

# 保存模型
save_model(model)

print("\n  模型训练完成!")

# =====
# 文件: sa_demo.py
# =====
"""
=====
文件名: sa_13.py
作者: zrd
功能: 算法演示配置 - 生成"优化前很差、优化后很好"的测试场景
描述: 提供演示数据生成、初始方案构造、匹配率计算和配置管理功能
=====
"""

import random
import json

```

```

from typing import List, Dict, Any, Tuple

DEMO_SCENARIOS = {
    "small": {
        "name": "小型演示 (8人, 3场地, 2时段)",
        "num_courts": 3,
        "num_users": 8,
        "num_time_slots": 2,
        "court_types": ["羽毛球", "乒乓球", "篮球"],
        "seed": 42,
        "expected_match_rate": 0.75,
        "description": "适合快速演示, 优化前后对比明显"
    },
    "medium": {
        "name": "中型演示 (16人, 5场地, 3时段)",
        "num_courts": 5,
        "num_users": 16,
        "num_time_slots": 3,
        "court_types": ["羽毛球", "乒乓球", "篮球", "网球"],
        "seed": 123,
        "expected_match_rate": 0.70,
        "description": "更接近真实场景, 效果显著"
    },
    "large": {
        "name": "大型演示 (30人, 8场地, 4时段)",
        "num_courts": 8,
        "num_users": 30,
        "num_time_slots": 4,
        "court_types": ["羽毛球", "乒乓球", "篮球", "网球", "足球"],
        "seed": 456,
        "expected_match_rate": 0.65,
        "description": "压力测试, 验证算法可扩展性"
    }
}

```

```

def generate_demo_data(
    num_courts: int = 5,
    num_users: int = 16,
    num_time_slots: int = 2,
    court_types: List[str] = None,
    seed: int = 42,
    bad_initial_ratio: float = 0.8
) -> Tuple[List[Dict], List[Dict], List[Dict]]:
    """

```

生成带时间维度的演示数据

关键设计:

1. 每个用户有明确的偏好类型 (1-2种)
2. 初始分配方案故意设置得很差 (大部分用户分配到不偏好的场地)

3. 算法应该能显著改善匹配率

Args:

num_courts: 场地数量
 num_users: 用户数量
 num_time_slots: 每个场地的时间段数
 court_types: 场地类型列表
 seed: 随机种子
 bad_initial_ratio: 初始分配错误比例 (0-1之间)

Returns:

(courts, users, time_slots) 用于算法输入
 """

```
random.seed(seed)
```

```
if court_types is None:
```

```
    court_types = ["羽毛球", "乒乓球", "篮球"]
```

```
time_slots = []
```

```
start_hour = 8
```

```
slot_duration = 1.5
```

```
for i in range(num_time_slots):
```

```
    start_time = start_hour + i * slot_duration
```

```
    end_time = start_time + slot_duration
```

```
    time_slots.append({
```

```
        "slot_id": i + 1,
```

```
        "date": "2026-08-16",
```

```
        "start_time": f"{int(start_time):02d}:{int((start_time % 1) * 60):02d}",
```

```
        "end_time": f"{int(end_time):02d}:{int((end_time % 1) * 60):02d}",
```

```
        "index": i
```

```
    })
```

```
courts = []
```

```
court_id = 1
```

```
for i in range(num_courts):
```

```
    court_type = court_types[i % len(court_types)]
```

```
    location = f"{chr(65 + i % 5)}区"
```

```
    for slot in time_slots:
```

```
        courts.append({
```

```
            "id": court_id,
```

```
            "court_name": f"{court_type}场地{chr(65 + i)}",
```

```
            "type": court_type,
```

```
            "location": location,
```

```
            "slot_id": slot["slot_id"],
```

```
            "date": slot["date"],
```

```
            "start_time": slot["start_time"],
```

```
            "end_time": slot["end_time"],
```

```
            "time_index": slot["index"],
```

```
            "court_index": i,
```

```

        "unique_key": f"court_{i}_slot_{slot['slot_id']}"
    })
    court_id += 1

users = []

type_counts = {}
for ct in court_types:
    type_counts[ct] = 0

for i in range(num_users):
    num_prefs = random.choices([1, 2], weights=[0.4, 0.6])[0]

    if i < num_users * 0.6:
        preferred_types = court_types[:2]
        if num_prefs == 1:
            prefs = [random.choice(preferred_types)]
        else:
            prefs = preferred_types.copy()
    else:
        prefs = random.sample(court_types, min(num_prefs, len(court_types)))

    time_prefs = random.sample(
        [s["slot_id"] for s in time_slots],
        min(random.randint(1, min(2, len(time_slots))), len(time_slots))
    ) if len(time_slots) > 0 else []

    for p in prefs:
        if p in type_counts:
            type_counts[p] += 1

    users.append({
        "id": i + 1,
        "name": f"用户{i + 1}",
        "preference": prefs if len(prefs) > 1 else (prefs[0] if prefs else ""),
        "time_preferences": time_prefs,
        "level": random.choice(["初级", "中级", "高级"]),
        "priority": random.randint(1, 5)
    })

return courts, users, time_slots

def create_bad_initial_plan(
    courts: List[Dict],
    users: List[Dict],
    time_slots: List[Dict],
    bad_ratio: float = 0.8
) -> List[Dict]:
    """

```

创建一个故意很差的分配方案，用于演示对比

Args:

courts: 场地列表
users: 用户列表
time_slots: 时间段列表
bad_ratio: 错误分配比例

Returns:

分配方案列表，每个元素包含 user_id, court_id, slot_id
"""

```
plan = []
num_users = len(users)
num_court_slots = len(courts)

users_with_prefs = []
users_without_prefs = []

for user in users:
    pref = user.get('preference', '')
    if pref:
        users_with_prefs.append(user)
    else:
        users_without_prefs.append(user)

court_types = {}
for court in courts:
    court_types[court['id']] = court.get('type', '')

user_pref_types = {}
for user in users:
    pref = user.get('preference', '')
    if isinstance(pref, list) and pref:
        user_pref_types[user['id']] = pref[0]
    elif isinstance(pref, str) and pref:
        user_pref_types[user['id']] = pref
    else:
        user_pref_types[user['id']] = None

used_courts = set()
court_slots = [c for c in courts]

for user in users_with_prefs:
    user_id = user['id']
    pref_type = user_pref_types.get(user_id)

    if random.random() < bad_ratio and pref_type:
        wrong_courts = [c for c in court_slots if c['id'] not in used_courts
                        and c.get('type', '') != pref_type]
        if wrong_courts:
```

```

        court = random.choice(wrong_courts)
    else:
        available = [c for c in court_slots if c['id'] not in used_courts]
        court = random.choice(available) if available else random.choice(court_slo...
    else:
        matching_courts = [c for c in court_slots if c['id'] not in used_courts
                           and c.get('type', '') == pref_type]
        if matching_courts:
            court = random.choice(matching_courts)
        else:
            available = [c for c in court_slots if c['id'] not in used_courts]
            court = random.choice(available) if available else random.choice(court_slo...

    used_courts.add(court['id'])
    plan.append({
        'user_id': user_id,
        'user_name': user.get('name', f'用户{user_id}'),
        'court_id': court['id'],
        'court_name': court.get('court_name', ''),
        'court_type': court.get('type', ''),
        'slot_id': court.get('slot_id', 1),
        'start_time': court.get('start_time', ''),
        'end_time': court.get('end_time', ''),
        'preference': pref_type
    })

for user in users_without_prefs:
    available = [c for c in court_slots if c['id'] not in used_courts]
    court = random.choice(available) if available else random.choice(court_slots)
    used_courts.add(court['id'])
    plan.append({
        'user_id': user['id'],
        'user_name': user.get('name', f"用户{user['id']}"),
        'court_id': court['id'],
        'court_name': court.get('court_name', ''),
        'court_type': court.get('type', ''),
        'slot_id': court.get('slot_id', 1),
        'start_time': court.get('start_time', ''),
        'end_time': court.get('end_time', ''),
        'preference': None
    })

return plan

def calculate_plan_match_rate(
    plan: List[Dict],
    users: List[Dict]
) -> Dict[str, Any]:
    """

```

计算一个分配方案的匹配率

Args:

plan: 分配方案

users: 用户列表（包含偏好信息）

Returns:

匹配统计信息

"""

```

user_prefs = {}
for user in users:
    pref = user.get('preference', '')
    if isinstance(pref, list):
        user_prefs[user['id']] = pref
    elif isinstance(pref, str) and pref:
        user_prefs[user['id']] = [pref]
    else:
        user_prefs[user['id']] = []

total = len(plan)
matches = 0
match_details = []

for item in plan:
    user_id = item.get('user_id')
    court_type = item.get('court_type', '')
    user_pref_list = user_prefs.get(user_id, [])

    is_match = court_type in user_pref_list if user_pref_list else False
    if is_match:
        matches += 1
    match_details.append({
        'user_id': user_id,
        'user_name': item.get('user_name', ''),
        'court_type': court_type,
        'preference': user_pref_list,
        'is_match': is_match
    })

return {
    'total': total,
    'matches': matches,
    'match_rate': matches / total if total > 0 else 0,
    'details': match_details
}

```

```

def get_demo_config(scenario_name: str = "medium") -> Dict[str, Any]:
    """

```

获取完整的演示配置

Args:

 scenario_name: 场景名称, 可选 'small', 'medium', 'large'

Returns:

 包含数据、初始方案、预期结果的完整配置

"""

if scenario_name not in DEMO_SCENARIOS:

 scenario_name = "medium"

scenario = DEMO_SCENARIOS[scenario_name]

courts, users, time_slots = generate_demo_data(

 num_courts=scenario['num_courts'],

 num_users=scenario['num_users'],

 num_time_slots=scenario['num_time_slots'],

 court_types=scenario['court_types'],

 seed=scenario['seed'],

 bad_initial_ratio=0.85

)

bad_plan = create_bad_initial_plan(

 courts, users, time_slots, bad_ratio=0.85

)

initial_stats = calculate_plan_match_rate(bad_plan, users)

return {

 'scenario': scenario,

 'courts': courts,

 'users': users,

 'time_slots': time_slots,

 'bad_plan': bad_plan,

 'initial_stats': initial_stats,

 'expected_match_rate': scenario['expected_match_rate']

}

def print_comparison(initial_stats: Dict, final_stats: Dict):

"""

打印优化前后对比

Args:

 initial_stats: 优化前统计

 final_stats: 优化后统计

"""

improvement = final_stats['match_rate'] - initial_stats['match_rate']

if improvement > 0.3:

 result_msg = "效果显著! 匹配率提升了 {:.1%}".format(improvement)

```

    elif improvement > 0.15:
        result_msg = "效果明显！匹配率提升了 {:.1%}".format(improvement)
    else:
        result_msg = "效果一般，匹配率提升了 {:.1%}".format(improvement)

    return {
        'initial_matches': initial_stats['matches'],
        'final_matches': final_stats['matches'],
        'initial_rate': initial_stats['match_rate'],
        'final_rate': final_stats['match_rate'],
        'total': initial_stats['total'],
        'improvement': improvement,
        'message': result_msg
    }

def test_demo_config() -> bool:
    """
    测试演示配置

    Returns:
        所有测试是否通过
    """
    all_passed = True
    for name in ['small', 'medium', 'large']:
        config = get_demo_config(name)

        if config['initial_stats']['match_rate'] >= 0.4:
            all_passed = False
        if config['expected_match_rate'] <= 0.5:
            all_passed = False

    return all_passed

def get_demo_data_for_api(scenario_name: str = "medium") -> Dict[str, Any]:
    """
    获取用于 API 的演示数据

    返回格式与 /api/optimize 接口的输入格式兼容
    """
    config = get_demo_config(scenario_name)

    courts_api = []
    for court in config['courts']:
        courts_api.append({
            "id": court.get('id', 0),
            "name": court.get('court_name', ''),
            "type": court.get('type', ''),
            "location": court.get('location', '')
        })

```

```
    })

    users_api = []
    for user in config['users']:
        pref = user.get('preference', '')
        if isinstance(pref, list):
            pref = pref[0] if pref else ''
        users_api.append({
            "id": user.get('id', 0),
            "username": user.get('name', ''),
            "preference": pref,
            "time_preferences": user.get('time_preferences', [])
        })

    return {
        "scenario": config['scenario']['name'],
        "courts": courts_api,
        "users": users_api,
        "time_slots": config['time_slots'],
        "bad_plan": config['bad_plan'],
        "initial_stats": config['initial_stats'],
        "expected_match_rate": config['expected_match_rate']
    }

if __name__ == "__main__":
    test_demo_config()
```